

`mpl_toolkits` isn't a single class or module with a fixed set of "methods."

It's a **namespace package** that contains multiple **subpackages, modules, classes, and functions**, and what's available depends on your **Matplotlib version**.

So there is no meaningful static list like "these are all the methods."

The correct way is to **inspect it programmatically**.

Below are the *right ways* to list everything.

1. What `mpl_toolkits` actually contains (high level)

Common subpackages include:

- `mpl_toolkits.mplot3d` – 3D plotting
- `mpl_toolkits.axes_grid1` – advanced axes/layout tools
- `mpl_toolkits.axisartist` – custom axis artists
- `mpl_toolkits.axes_grid` (legacy)

Each of these then contains **classes and functions** (the "methods" you're probably referring to).

2. List submodules inside `mpl_toolkits`

```
import mpl_toolkits
import pkgutil

[submodule.name for submodule in pkgutil.iter_modules(mpl_toolkits.__path__)]
```

Example output (varies by version):

```
['axes_grid', 'axes_grid1', 'axisartist', 'mplot3d']
```

3. List everything inside a specific submodule

Example: `mplot3d`

```
from mpl_toolkits import mplot3d
dir(mplot3d)
```

This lists **all names** (classes, functions, constants).

4. List public classes & functions only

```
import inspect
from mpl_toolkits.mplot3d import axes3d

members = inspect.getmembers(axes3d)

public = [
    name for name, obj in members
    if not name.startswith("_")
]

public
```

5. List methods of a specific class (most common need)

Example: Axes3D

```
from mpl_toolkits.mplot3d import Axes3D
import inspect

[name for name, obj in inspect.getmembers(Axes3D, inspect.isfunction)]
```

This will return methods like:

- plot
 - scatter
 - plot_surface
 - plot_wireframe
 - contour3D
 - set_zlabel
 - etc.
-

6. Dump *everything* recursively (advanced)

```
import pkgutil
import importlib
import inspect
import mpl_toolkits

for m in pkgutil.walk_packages(mpl_toolkits.__path__, mpl_toolkits.__name__ +
    "."):
    try:
        module = importlib.import_module(m.name)
        print(f"\nMODULE: {m.name}")
        for name, obj in inspect.getmembers(module):
            if not name.startswith("_"):
```

```
        print(" ", name)
except Exception:
    pass
```

⚠ This will produce a lot of output.

Summary

- There is **no fixed “list of methods”** for `mpl_toolkits`
- It’s a **package of subpackages**
- You must inspect:
 - submodules → `pkgutil.iter_modules`
 - module contents → `dir()`
 - class methods → `inspect.getmembers`

Below is a **structured list of what’s in `mpl_toolkits.mplot3d`** (the *module*, not just `Axes3D`). This covers the **public classes, functions, and submodules** you actually use.



`mpl_toolkits.mplot3d` — top-level exports

```
from mpl_toolkits import mplot3d
dir(mplot3d)
```

The important public items are:



Core classes

Axes

- **Axes3D** – the 3D Axes class (main entry point)

Art / collections

- Line3D
 - Line3DCollection
 - Patch3D
 - Patch3DCollection
 - Poly3DCollection
 - Path3DCollection
-



Projection & transforms

- proj3d
 - proj_transform
 - inv_transform
 - world_transformation
-



Plot helpers / internal modules (public but advanced)

Submodules

- axes3d – implementation of Axes3D
- art3d – 3D artist conversions
- proj3d – $3D \rightarrow 2D$ projection math

You can access them as:

```
from mpl_toolkits.mplot3d import art3d, axes3d, proj3d
```



art3d (commonly used helpers)

From `mpl_toolkits.mplot3d.art3d`:

- line_2d_to_3d
- patch_2d_to_3d
- pathpatch_2d_to_3d
- text_2d_to_3d
- set_3d_properties

Example:

```
from mpl_toolkits.mplot3d.art3d import line_2d_to_3d
```



Collection classes (used internally but accessible)

- `Line3DCollection`
- `Poly3DCollection`
- `Path3DCollection`
- `Patch3DCollection`

These are useful for **custom 3D artists** and performance-heavy plots.



Utility / configuration

- `get_test_data` (demo data helper)
-



Inheritance note (important)

`Axes3D` inherits from:

`matplotlib.axes.Axes`

So **most 2D Axes methods are available**, even though only some make sense in 3D.



Full introspection (exact to your version)

```
import inspect
from mpl_toolkits import mplot3d

[name for name, obj in inspect.getmembers(mplot3d) if not
name.startswith("_")]
```



Practical usage pattern

```
from mpl_toolkits.mplot3d import Axes3D # registers projection
fig.add_subplot(111, projection="3d")
```

(Importing `Axes3D` is still required for side-effects.)

Summary

mplot3d provides:

- Axes3D (main API)
- 3D artist & collection classes
- Projection math (proj3d)
- Artist conversion helpers (art3d)

Here is a **clean, practical list of the public methods on `mpl_toolkits.mplot3d.Axes3D`**, grouped by purpose.
(This reflects standard Matplotlib 3.x behavior; exact count may vary slightly by version.)

Core plotting methods (3D)

- `plot`
 - `scatter`
 - `bar`
 - `bar3d`
 - `plot_surface`
 - `plot_wireframe`
 - `plot_trisurf`
 - `contour`
 - `contourf`
 - `contour3D`
 - `tricontour`
 - `tricontourf`
 - `voxels`
 - `quiver`
 - `stem`
 - `plot_date`
 - `imshow` (limited 3D support)
-

Axis / limits / labels

- `set_xlabel`
 - `set_ylabel`
 - `set_zlabel`
 - `get_xlabel`
 - `get_ylabel`
 - `get_zlabel`
 - `set_xlim`
 - `set_ylim`
 - `set_zlim`
 - `get_xlim`
 - `get_ylim`
 - `get_zlim`
 - `set_xlim3d`
 - `set_ylim3d`
 - `set_zlim3d`
 - `set_box_aspect`
 - `set_aspect`
-



View & camera control

- `view_init`
 - `invert_xaxis`
 - `invert_yaxis`
 - `invert_zaxis`
 - `get_proj`
 - `dist` (attribute controlling camera distance)
-



Ticks, grids, panes

- `set_xticks`
- `set_yticks`
- `set_zticks`
- `get_xticks`
- `get_yticks`
- `get_zticks`
- `set_xticklabels`
- `set_yticklabels`
- `set_zticklabels`
- `grid`
- `tick_params`
- `xaxis`
- `yaxis`
- `zaxis`



Collections & artists

- `add_collection`
- `add_collection3d`
- `add_line`
- `add_patch`
- `text`
- `text2D`



Styling & appearance

- `set_title`
- `set_facecolor`
- `set_frame_on`
- `set_axis_on`
- `set_axis_off`
- `cla` (clear axes)
- `clear`



Internal / advanced (public but rarely used)

- `auto_scale_xyz`
- `get_w_lims`
- `mouse_init`
- `format_coord`
- `draw`



Inherited from `matplotlib.axes.Axes`

`Axes3D` inherits many 2D Axes methods, including:

- `legend`
- `annotate`
- `axhline, axvline`
- `margins`
- `set_prop_cycle`

- `callbacks`
- `contains`
- `get_children`

(These work mostly as expected, but not all make sense in 3D.)

How to verify on your system

```
from mpl_toolkits.mplot3d import Axes3D
import inspect

[name for name, obj in inspect.getmembers(Axes3D) if not
name.startswith("_")]
```